DAM eta DAW ZIKLOETAN

APUNTES PRÁCTICOS

{ JSON – Javascript Object Notation }

J. Koldobika Arriaga B.

Ikasgaia
Lenguajes de Marcas y Sistemas de Gestión de Información

Zikloa
Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)

Kurtsoa
1º Maila

Gaia
Formato de intercambio de datos: JSON

Irakaslea
J. Koldobika Arriaga

Indice de Contenido

1 Qué es JSON....2
1.1 ¿Por qué se usa JSON?....2
1.2 JSON vs XML: una comparativa....2
2 Reglas y Sintaxis....3
2.1 Las 6 reglas fundamentales....3
2.2 Estructura básica de un fichero JSON....3
3 Tipos de Datos JSON....4
3.1 Detalle de cada tipo....4
3.1.1 String (cadena de texto)....4
3.1.2 Number (número)....4
3.1.3 Boolean y Null....5
4 Estructuras y Objetos en JSON....6
4.1 Objetos JSON....6
4.2 Arrays JSON....6
4.2.1 JSON Anidado (Nested JSON)....6
5 Casos de Uso en el Mundo Real....8
5.1 APIs REST (el uso más frecuente)....8
5.2 Ficheros de configuración....8
5.3 Almacenamiento de datos (NoSQL)....9
5.4 Exportación e importación de datos....9
5.5 Comunicación entre microservicios....9
6 Parseando y Generando JSON....10
6.1 En JavaScript / Node.js....10
6.2 En Python....10
6.3 Manejo de errores al parsear....11
6.4 Herramientas útiles para trabajar con JSON....11
7 Mejores Prácticas para Principiantes....12
7.1 Resumen Visual de Sintaxis....13

1 Qué es JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos. Es un estándar abierto basado en texto que representa datos estructurados de forma legible tanto para humanos como para máquinas.

Aunque su nombre incluye 'JavaScript', JSON es completamente independiente del lenguaje de programación. Hoy en día es compatible de forma nativa con Python, Java, C#, PHP, Ruby, Go, Kotlin y prácticamente cualquier lenguaje moderno.

> 💡 **Dato histórico**
> JSON fue especificado por Douglas Crockford a principios de los años 2000 y está descrito en el estándar RFC 8259. Antes de JSON, el formato más utilizado para el intercambio de datos era XML, pero JSON se impuso por ser mucho más sencillo y compacto.

1.1 ¿Por qué se usa JSON?

JSON se ha convertido en el estándar de facto para el intercambio de datos en aplicaciones web, APIs REST y ficheros de configuración gracias a estas ventajas:

- **Legibilidad**: su sintaxis es clara e intuitiva, muy fácil de leer y entender.
- **Ligereza**: genera ficheros más compactos que XML, reduciendo el ancho de banda consumido.
- **Velocidad de parseo**: los intérpretes JSON son rápidos y están integrados en los navegadores.
- **Universalidad**: compatible con todos los lenguajes de programación modernos.
- **Soporte nativo en JavaScript**: los navegadores lo procesan de forma directa sin librerías externas.

1.2 JSON vs XML: una comparativa

Para ilustrar las diferencias, observa cómo se representa el mismo dato en ambos formatos:

XML (verbose)	JSON (compacto)

`<alumno>`
`  <nombre>Ana</nombre>`
`  <edad>20</edad>`
`  <activo>true</activo>`
`</alumno>`
	`{`
	`  "nombre": "Ana",`
	`  "edad": 20,`
	`  "activo": true`
	`}`

2 Reglas y Sintaxis

JSON tiene unas reglas sintácticas muy estrictas y concretas. Si se incumple cualquiera de ellas, el fichero JSON se considera inválido y los parsers lanzarán un error.

2.1 Las 6 reglas fundamentales

1. **Estructura base**: Los datos se organizan en pares clave-valor.
2. **Comillas dobles obligatorias**: Las claves SIEMPRE deben ir entre comillas dobles (""): "nombre", "edad".
3. **Separación por comas**: Los elementos se separan por comas, EXCEPTO el último elemento del bloque.
4. **Delimitadores**: Los objetos se delimitan con llaves { } y los arrays con corchetes [ ].
5. **Sin comentarios:** JSON no admite comentarios (// ni /* */). El estándar no los contempla.
6. **Case-sensitive:** JSON distingue entre mayúsculas y minúsculas: "Nombre" y "nombre" son claves diferentes.

2.2 Estructura básica de un fichero JSON

Un fichero JSON válido debe contener exactamente UN valor raíz: un objeto o un array.

```
// Ejemplo de objeto JSON válido completo:
{
    "nombre"  : "María García",
    "edad"    : 22,
    "activo"  : true,
    "notas"   : [8.5, 9.0, 7.5],
    "direccion": {
        "calle"  : "Calle Mayor, 15",
        "ciudad" : "Madrid",
        "cp"     : "28001"
    },
    "telefono": null
}
```

⚠ **Errores sintácticos más comunes**

1) Usar comillas simples en lugar de dobles: {'nombre': 'Ana'} es INCORRECTO. 2) Dejar una coma después del último elemento: {"a": 1, "b": 2,} es INCORRECTO. 3) Intentar añadir comentarios: { // esto es un comentario } es INCORRECTO.

3 Tipos de Datos JSON

JSON soporta exactamente 6 tipos de datos primitivos. Conocerlos con precisión es fundamental para construir y validar estructuras JSON correctamente.

Tipo	Ejemplo en JSON	Descripción
String	`"Hola Mundo"`	Texto entre comillas dobles. Admite caracteres Unicode y secuencias de escape.
Number	42 / 3.14 / -7	Enteros y decimales. NO se usan comillas. No existe distinción int/float.
Boolean	true / false	Solo dos valores posibles, siempre en minúsculas.
Null	null	Representa la ausencia de valor. Siempre en minúsculas.
Object	`{ "clave": valor }`	Colección desordenada de pares clave-valor delimitada por { }.
Array	[1, "hola", true]	Lista ordenada de valores de cualquier tipo, delimitada por [ ].

3.1 Detalle de cada tipo

3.1.1 String (cadena de texto)

Los strings en JSON van siempre entre comillas dobles. Admiten secuencias de escape para incluir caracteres especiales:

```
{ "mensaje"     : "Hola, mundo!",
  "ruta"        : "C:\\Users\\alumno\\fichero.json",
  "tabulado"    : "Columna1\tColumna2",
  "con_comilla": "El profesor dijo \"Atención\"" }
```

Secuencias de escape disponibles: \" (comilla doble), \\ (barra inversa), \n (nueva línea), \t (tabulador), \uXXXX (carácter Unicode).

3.1.2 Number (número)

JSON no distingue entre enteros y decimales. Los formatos válidos son:

```
{
  "entero"    : 42,
  "negativo"  : -15,
  "decimal"   : 3.14159,
  "notacion"  : 1.5e10,
```

```
  "negExp"     : 2.5E-4
}
```

⚠ **Atención con los números**
JSON NO soporta Infinity, NaN ni números octales/hexadecimales. Tampoco se deben usar comillas en los números: "edad": "22" convierte el número en texto.

3.1.3 Boolean y Null

Deben escribirse SIEMPRE en minúsculas. Son case-sensitive:

```
{
  "matriculado" : true,
  "aprobado"    : false,
  "telefono"    : null
}
```

💡 **Buena práctica con null**
Usa null cuando un campo existe en la estructura pero no tiene valor en ese momento. Es diferente a omitir la clave por completo: { "telefono": null } indica que el campo existe pero está vacío; {} indica que el campo no se ha definido.

4 Estructuras y Objetos en JSON

Los objetos y los arrays son los contenedores que dan a JSON su verdadero poder. Pueden anidarse en cualquier nivel de profundidad para modelar datos del mundo real.

4.1 Objetos JSON

Un objeto JSON es una colección no ordenada de pares "clave": valor, delimitada por llaves { }. Cada clave debe ser única dentro del mismo objeto.

```
// Objeto JSON: información de un alumno
{
    "id"          : 101,
    "nombre"      : "Carlos López",
    "curso"       : "1DAM",
    "nota_media" : 7.85,
    "becado"      : true
}
```

4.2 Arrays JSON

Un array JSON es una colección ordenada de valores, delimitada por corchetes [ ]. Los elementos pueden ser de cualquier tipo y pueden mezclarse tipos distintos en el mismo array (aunque no es la práctica recomendada).

```
// Array de strings
[ "Lunes", "Martes", "Miércoles", "Jueves", "Viernes" ]

// Array de números (notas)
[ 5.5, 7.0, 8.5, 6.0, 9.5 ]

// Array de objetos (lista de alumnos)
[
    { "id": 1, "nombre": "Ana",    "nota": 9.0 },
    { "id": 2, "nombre": "Luis",   "nota": 7.5 },
    { "id": 3, "nombre": "Sofía",  "nota": 8.2 }
]
```

4.2.1 JSON Anidado (Nested JSON)

Los objetos y arrays pueden anidarse entre sí de forma ilimitada. Este es el mecanismo que permite modelar jerarquías y relaciones complejas de datos:

```
// Estructura anidada: un alumno con módulos y dirección
{
    "id"       : 201,
    "nombre"   : "Elena Martínez",
    "direccion": {
```

```
        "calle"  : "Av. Constitución, 45",
        "ciudad" : "Sevilla",
        "cp"     : "41001",
        "coordenadas": {
            "lat": 37.3886,
            "lon": -5.9823
        }
    },
    "modulos": [
        { "nombre": "Lenguajes de Marcas", "horas": 96,  "nota": 8.0 },
        { "nombre": "Programación",        "horas": 256, "nota": 7.5 },
        { "nombre": "Bases de Datos",      "horas": 192, "nota": 9.0 }
    ]
}
```

💡 **Consejo de diseño**

Evita anidar más de 3 o 4 niveles de profundidad. Una estructura muy profunda es difícil de leer y mantener. Si la estructura se vuelve muy compleja, es señal de que quizás necesitas rediseñar el modelo de datos.

5 Casos de Uso en el Mundo Real

JSON está presente en prácticamente todos los sistemas de software modernos. Estos son los contextos donde lo encontrarás con más frecuencia durante tu carrera profesional:

5.1 APIs REST (el uso más frecuente)

Cuando una aplicación móvil, web o de escritorio necesita datos de un servidor, utiliza una API REST. Tanto la petición como la respuesta viajan en formato JSON. Este es el escenario donde más utilizarás JSON en DAM.

```
// Respuesta de una API REST al pedir datos de un producto
// GET https://api.tienda.com/productos/42
{
    "status"   : "success",
    "codigo"   : 200,
    "producto" : {
        "id"          : 42,
        "nombre"      : "Teclado Mecánico RGB",
        "precio"      : 89.99,
        "disponible" : true,
        "categorias" : ["Periféricos", "Gaming", "Teclados"]
    }
}
```

5.2 Ficheros de configuración

JSON se usa ampliamente para configurar herramientas de desarrollo. Por ejemplo, el fichero "package.json" en proyectos Node.js define las dependencias y scripts del proyecto:

```
// package.json — configuración de un proyecto Node.js
{
    "name"        : "mi-aplicacion-dam",
    "version"     : "1.0.0",
    "description" : "Proyecto de Lenguajes de Marcas",
    "scripts"     : {
        "start" : "node index.js",
        "test"  : "jest"
    },
    "dependencies": {
        "express" : "^4.18.0",
        "axios"   : "^1.4.0"
    }
}
```

5.3 Almacenamiento de datos (NoSQL)

Bases de datos NoSQL como MongoDB o Firebase almacenan los datos directamente en formato similar a JSON (BSON/JSON). Cada registro de la base de datos es un documento JSON, lo que facilita trabajar con datos sin esquemas fijos.

5.4 Exportación e importación de datos

Las aplicaciones permiten exportar sus datos a JSON para que puedan ser importados por otras aplicaciones. Es el equivalente moderno al CSV, pero con soporte para estructuras complejas y jerarquías.

5.5 Comunicación entre microservicios

En arquitecturas de microservicios, los distintos componentes de una aplicación se comunican entre sí mediante mensajes JSON a través de colas de mensajería o llamadas HTTP directas.

6 Parseando y Generando JSON

"Parsear" un JSON significa convertir una cadena de texto en formato JSON en una estructura de datos nativa del lenguaje de programación (un objeto, un diccionario, etc.). "Serializar" o "stringify" es el proceso inverso: convertir una estructura de datos en una cadena de texto JSON.

6.1 En JavaScript / Node.js

```
// ── PARSEAR: string JSON → objeto JavaScript ────────────────
const jsonString = '{"nombre": "Lucía", "edad": 21, "activo": true}';
const objeto = JSON.parse(jsonString);
console.log(objeto.nombre);  // Salida: Lucía
console.log(objeto.edad);    // Salida: 21

// ── GENERAR: objeto JavaScript → string JSON ────────────────
const alumno = { nombre: "Lucía", edad: 21, activo: true };
const jsonResultante = JSON.stringify(alumno);
// Resultado: '{"nombre":"Lucía","edad":21,"activo":true}'

// ── GENERAR con formato legible (indentado) ─────────────────
const jsonFormateado = JSON.stringify(alumno, null, 2);
// El segundo argumento (null) es un filtro, el tercero (2) son los espacios
```

6.2 En Python

```
import json

# ── PARSEAR: string JSON → diccionario Python ────────────────
json_string = '{"nombre": "Lucía", "edad": 21, "activo": true}'
diccionario = json.loads(json_string)
print(diccionario["nombre"])  # Salida: Lucía

# ── LEER desde un fichero .json ──────────────────────────────
with open("datos.json", "r", encoding="utf-8") as f:
    datos = json.load(f)

# ── GENERAR: diccionario Python → string JSON ────────────────
alumno = {"nombre": "Lucía", "edad": 21, "activo": True}
json_resultante = json.dumps(alumno, ensure_ascii=False, indent=2)

# ── ESCRIBIR en un fichero .json ─────────────────────────────
with open("salida.json", "w", encoding="utf-8") as f:
    json.dump(alumno, f, ensure_ascii=False, indent=2)
```

6.3 Manejo de errores al parsear

Cuando recibimos un JSON de una fuente externa (una API, un fichero), es imprescindible manejar los posibles errores de parseo. Un JSON malformado lanzará una excepción:

```
// JavaScript — manejo de errores con try/catch
try {
    const datos = JSON.parse(textoRecibido);
    console.log("JSON válido:", datos);
} catch (error) {
    console.error("Error: el JSON recibido no es válido:", error.message);
}
```

6.4 Herramientas útiles para trabajar con JSON

- **Validación:** JSONLint (jsonlint.com): validador online que detecta errores sintácticos.
- **Extensiones:** JSON Formatter & Validator: extensiones para navegadores Chrome y Firefox.
- **Prueba de APIs:** Postman: herramienta para probar APIs REST y visualizar respuestas JSON.
- **Terminal:** jq: herramienta de línea de comandos para filtrar y transformar JSON.
- **Editor:** VS Code: editor con resaltado de sintaxis y formateo automático de JSON.

7 Mejores Prácticas para Principiantes

Adoptar buenas prácticas desde el principio te ahorrará muchos errores y hará que tu código sea más profesional y fácil de mantener. Aquí tienes las más importantes:

01

Valida siempre tu JSON antes de usarlo

Antes de integrar un JSON en tu código, pásalo por un validador como JSONLint. Un solo carácter mal puesto (una coma de más, una comilla simple en vez de doble) hace que todo el documento sea inválido.

02

Usa nombres de claves consistentes y descriptivos

Elige una convención de nombres y mantente fiel a ella. Lo más habitual en JSON es el camelCase (nombreCompleto, fechaNacimiento). Evita abreviaturas oscuras: "fc_nac" es peor que "fechaNacimiento".

03

Mantén la consistencia de tipos en los arrays

Si un array contiene objetos, todos los objetos deberían tener la misma estructura. Mezclar tipos distintos en un array (números, strings, objetos) dificulta el procesamiento y genera confusión.

04

Usa null en lugar de strings vacíos cuando no hay valor

Para representar la ausencia de valor, usa null en lugar de "" (cadena vacía) o -1. Esto es semánticamente más correcto y permite distinguir "campo vacío" de "campo con valor vacío".

05

Formatea el JSON al escribirlo, minifícalo al enviarlo

Durante el desarrollo, usa JSON indentado (con espacios) para facilitar la lectura y depuración. Al enviar datos por la red (producción), usa JSON sin espacios (minificado) para reducir el tamaño.

06

No almacenes contraseñas ni datos sensibles sin cifrar en JSON

Un fichero JSON es texto plano y puede ser leído por cualquiera que acceda al fichero o a la red. Nunca guardes contraseñas, tokens o datos personales sensibles sin las medidas de seguridad adecuadas.

7.1 Resumen Visual de Sintaxis

```
// ✅ JSON CORRECTO
{
    "alumno"   : "Ana García",       // ✅ Comillas dobles en clave y valor
string
    "edad"     : 20,                 // ✅ Número sin comillas
    "aprobado" : true,               // ✅ Boolean en minúsculas
    "telefono" : null                // ✅ null en minúsculas, sin coma al
final
}

// ❌ JSON INCORRECTO — errores frecuentes
{
    'alumno'   : 'Ana García',       // ❌ Comillas simples no válidas
    "edad"     : "20",               // ❌ Número entre comillas → es string
    "aprobado" : True,               // ❌ Boolean con mayúscula
    "telefono" : Null,               // ❌ Null con mayúscula
    "extra"    : undefined,          // ❌ undefined no existe en JSON
}
```